Sensitive Data Exposure Prevention in PHP

1. If web applications don't protect sensitive data such as financial information and passwords, attackers can gain access to these data and sell or utilize it for nefarious purposes. One popular method for stealing sensitive information is using a man-in-the-middle attack.

2. Data exposure risk can be minimized by encrypting all sensitive data as well as disabling the caching of any sensitive information. Additionally, web application developers should take care to ensure that they are not unnecessarily storing any sensitive data.

Prevention:

1. Checking type of password hash algorithms inside database:

a. There are plenty of hash algorithms in the security world, however there are some that are weak and outdated and shouldn't be used.

b. ASST recognizes stored password formats such as MD5, SHA-1, SHA-256, SHA-384 and SHA-512 (fast hashes, not suitable for passwords) and bcrypt and Argon2 (suitable).

c. It uses the length and prefix of the stored value to recognize the format; the values themselves are never read.
   i. MD5 16 bytes hash algorithm = 32 chars
   ii. SHA-1 20 bytes hash algorithm = 40 chars
   iii. BCRYPT 30 bytes salt+hash algorithm = 60 chars
   iv. SHA-256 32 bytes hash algorithm = 64 chars
   v. SHA-384 48 bytes hash algorithm = 96 chars
   vi. SHA-512 64 bytes hash algorithm = 128 chars

d. Notes:
   i. General-purpose hashes are fast, so attackers can try billions of passwords per second whatever their length. Use password_hash() (bcrypt or Argon2id), which is deliberately slow and adds a random salt.
   ii. some other algorithms have the same char length, but they have different specifications, but we will concentrate on the length here.
   iii. Short values that do not look like a hash are reported as possible plain-text passwords.

e. password_hash() stores a unique random salt inside the hash, so no separate salt column is needed.

f. Use password_verify() to check passwords and password_needs_rehash() to upgrade older hashes at the next login.

To read more about hashes, go here:
https://www.php.net/manual/en/function.hash.php

2. Make sure HTTPS is used by force redirecting HTTP to HTTPS, using (.htaccess)

a. First check the existence of the ".htaccess" in the project root
b. Then write the following lines in it if not already written:
    - RewriteEngine On
    - RewriteCond %{HTTPS} off
    - RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
c. Above lines of code redirect clients from HTTP to HTTPS. Add the HSTS header as well: Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"

    To read more about htaccess redirection, go here:
    https://www.freecodecamp.org/news/how-to-redirect-http-to-https-using-htaccess/

All Above Explained ideas will be Scanned in the Toolkit.

3. Some Verbal Advices

a. Use HTTPS with TLS 1.2 and TLS 1.3 only; these use ephemeral key exchange, which provides forward secrecy.
b. Implement HSTS (HTTP Strict Transport Security): to Force HTTPS on browsers and Hackers cannot override the first HTTP redirect to HTTPS
c. Encrypt sensitive data such as Credit Card info and SSN and NOT TO Store Encryption Keys with them.
d. Optionally add a pepper (a secret key kept outside the database) with hash_hmac() before password_hash()

To read more about Sensitive Data Exposure:
https://www.youtube.com/watch?v=2RKbacrkUBU
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/